```
    print "{$file} : {$line}" . PHP_EOL;

    return true;
}, E_WARNING | E_USER_NOTICE);

print $undef;
trigger_error("TRIGGERED NOTICE", E_USER_NOTICE);
trigger_error("TRIGGERED WARNING", E_USER_WARNING);

Output:
WARNING!!! Undefined variable $undef
/usr/src/myapp/error_handler.php : 18
Just a notice... TRIGGERED NOTICE
/usr/src/myapp/error_handler.php : 19
PHP Warning:  TRIGGERED WARNING in /usr/src/myapp/05.03_set_error_handler.
php on line 20
```

Here we've used an anonymous function, which will be passed four arguments when an error is emitted. In order, these are the integer value of the error level, the error message itself, the full file path of the offending code, and the line number that triggered the error. We use the `$errno` argument with `switch` to determine which logic we want (just simple `print` statements in this case, but I'm sure you can imagine the possibilities), then the remaining message, file path, and line number arguments to form the rest of our message. After the callable, we then pass an error-level expression: we've set our handler to trigger on `E_WARNING` and `E_USER_NOTICE`.

Next, we cause some problems: an attempt to use an undefined variable (always an `E_WARNING` level error in PHP 8) and two calls to `trigger_error()` with levels of `E_USER_NOTICE` and `E_USER_WARNING`, respectively. You can see from the output that our handler is invoked for `E_WARNING` and `E_USER_NOTICE`, and the default handler is invoked for the `E_USER_WARNING`.

A user-defined error handler doesn't need to output any messages at all; the default handler can do that for you, and this is just an exercise. You could choose to do something more unusual, like, say, set a global flag to a new value, which would provide a mechanism for affecting the rest of the program's logic… but don't do that. Errors are a bit of a blunt instrument for controlling your program; there's a better way.